

Минимални покриващи дървета.

Алгоритми на Prim и Kruskal

1. Дефиниция на задачата

1.1 Какъв тип графи разглеждаме?

Разглеждаме **неориентирани, свързани, тегловни графи**. Нека $G = (V, E)$ е такъв граф, а $w : E \rightarrow \mathbb{R}$ е **тегловна функция** върху ребрата. На всяко ребро $e \in E$ съответства число $w(e)$ — теглото му.

Защо именно такива графи?

- **Неориентирани:** МПД като понятие има смисъл само за неориентирани графи. Обобщението върху ориентирани графи (Minimum Equivalent Digraph) е съвсем друга задача — и при това е NP-трудна. Неориентираността е съществена, защото идеята "ребро между две части на графа" е симетрична.
- **Свързан:** ако графът не е свързан, той няма покриващо дърво изобщо (Наблюдение 66 — граф има поне едно покриващо дърво тстк е свързан). Тогава задачата би била за **минимална покриваща гора** — по едно дърво за всяка компонента.
- **Тегловни:** целта е оптимизация по сума от тегла; без тегла задачата би се сведала до намиране на което и да е покриващо дърво, което е тривиално с BFS/DFS.

1.2 Покриващо дърво и МПД

Определение (покриващо дърво). Покриващ подграф на $G = (V, E)$ е всеки подграф $G' = (V, E')$, който съдържа всички върхове на G . **Покриващо дърво** (ПД) е покриващ подграф, който е дърво.

Интуитивно: ПД "докосва" всеки връх и е без цикли — точно $n - 1$ ребра (където $n = |V|$).

Определение (тегло на ПД). Ако T е ПД, теглото му е: $w(T) = \sum_{e \in E(T)} w(e)$

Определение (МПД). T' е **минимално покриващо дърво** (МПД), ако за всяко ПД T'' е вярно, че $w(T') \leq w(T'')$.

⚠ МПД не е задължително уникално. Може да има няколко различни ПД с едно и също минимално тегло.

1.3 Формална формулировка

Изч. задача 26 (Minimum Spanning Tree, MST):

- **Екземпляр:** наредена двойка $\langle G, w \rangle$ — неориентиран свързан тегловен граф.
- **Решение:** минимално покриващо дърво T на G .

Това е **оптимизационна** задача в **тежък вариант** — искаме самото дърво, а не само теглото му.

1.4 Интуиция: защо изобщо ни е МПД?

Класически примери: проектиране на електрическа мрежа, която свързва всички градове с минимална обща дължина кабели; план за полагане на оптика; намиране на "скелет" на граф за приближено решаване на по-сложни задачи (TSP-приближения и др.).

2. Сигурно ребро. Теорема за съгласуваното множество (МПД-теорема)

Преди да говорим за алгоритми, ни трябва **математическата гаранция**, че алчните локални решения водят до глобално оптимално решение.

2.1 Обща схема на алгоритъм за МПД

```
Обща схема на алгоритъм за МПД( $G, w$ ):  
1   $A \leftarrow \emptyset$   
2  while  $A$  не образува ПД do  
3      намери ребро  $e$ , което е сигурно за  $A$   
4       $A \leftarrow A \cup \{e\}$   
5  return  $A$ 
```

Във всеки момент A е подмножество на ребрата на някое МПД (така започваме и така поддържаме инварианта). while-цикълът се изпълнява точно $n - 1$ пъти, защото ПД има $n - 1$ ребра.

Цялата дълбочина е в ред 3: как да намерим сигурно ребро?

2.2 Ключови определения

Определение 91 (сигурно ребро). Нека $A \subset E(G)$, такова че съществува МПД T с $A \subset E(T)$. Реброто $e \in E(G) \setminus A$ е **сигурно за A** , ако съществува МПД T' , такова че $A \cup \{e\} \subseteq E(T')$.

Интуиция: "сигурно ребро" е такова, което не разваля възможността да достигнем някакво МПД. Ако A са ребрата, които сме сложили в кофата си досега, и тази кофа все още е в "правилния път" към някое МПД, то e е сигурно, ако след добавянето му пак сме в правилния път.

Определение 92 (срез). Срез на G е разбиване $S = \{V_1, V_2\}$ на $V(G)$ на два непразни дяла.

- Ребро $e = (u, v)$ **прекосява S** , ако $u \in V_1$ и $v \in V_2$.
- Срезът S е **съобразен с $E' \subseteq E(G)$** , ако нито едно ребро от E' не прекосява S .
- В тегловен граф ребро, прекосяващо S , е **леко**, ако има минимално тегло измежду всички ребра, прекосяващи S .

Интуиция: срезът е "стена", която разделя графа на две части. Лекото ребро е "най-евтината врата" в тази стена. "Съобразен с E' " означава: всички ребра в E' са изцяло от едната страна на стената — стената "не ги пресича".

2.3 МПД-теорема (Теорема 94)

Теорема 94 (МПД-теорема). Нека $G = (V, E)$ е свързан тегловен граф. Нека $A \subset E$ е такова, че съществува МПД T с $A \subset E(T)$. Нека $S = \{V_1, V_2\}$ е срез, **съобразен с A** . Нека $e = (u, v)$ е произволно леко ребро, прекосяващо S . **Тогава e е сигурно за A .**

2.4 Доказателство на МПД-теоремата (подробно, с интуиция)

Целта: да покажем, че съществува МПД T' , в което се съдържат и A , и e .

Случай 1: $e \in E(T)$. Тогава T самото е МПД, съдържащо $A \cup \{e\}$, и сме готови. ✓

Случай 2: $e \notin E(T)$. Тук е цялата работа. Стъпките:

Стъпка 1: Добавяме e към T . Получаваме граф $U = T + e$. Тъй като T е дърво (свързано и без цикли) и добавяме едно ребро, в U се образува **точно един цикъл** — нарича се **унициклически граф**. Нека този цикъл е C . Очевидно $e \in E(C)$.

Защо точно един цикъл? Защото в дървото между u и v има единствен път; добавянето на (u, v) затваря точно него.

Стъпка 2: Намираме друго ребро в C , което също прекосява S . Цикълът C преминава през среза S — започва от $u \in V_1$, отива до $v \in V_2$ и се връща. Всеки път от V_1 до V_2 и обратно прекосява среза четен брой пъти. Реброто e е едно "прекосяване". Понеже цикълът се връща в V_1 , **трябва да има поне още едно ребро $e' \neq e$ в C , което прекосява S .**

Стъпка 3: $e' \notin A$. Това е важно! Срезът S е съобразен с A — никое ребро от A не го прекосява. Но e' прекосява S . Следователно $e' \notin A$.

Стъпка 4: Заменяме e' с e . Премахваме e' от U , получаваме граф $T' = T + e - e'$. Това отново е покриващо дърво:

- $|E(T')| = |E(T)| + 1 - 1 = n - 1$
- T' е свързан (премахването на ребро от единствения цикъл запазва свързаността).
- T' няма цикли (премахнахме единственото ребро от единствения цикъл).

Стъпка 5: Сравнение на теглата. $w(T') = w(T) + w(e) - w(e')$

Тъй като e и e' прекосяват S , а e е **леко** ребро на S : $w(e) \leq w(e') \implies w(e) - w(e') \leq 0 \implies w(T') \leq w(T)$

Но T е МПД, значи $w(T) \leq w(T')$ по дефиниция. Следователно $w(T) = w(T')$, тоест T' също е МПД.

Стъпка 6: T' съдържа $A \cup \{e\}$.

- $A \subset E(T)$, а ребрата от A не прекосяват S , докато e' прекосява S . Значи $e' \notin A$. Следователно премахването на e' не отнема никое ребро от A : $A \subset E(T')$.
- $e \in E(T')$ по конструкция.

Следователно $A \cup \{e\} \subseteq E(T')$. ■

2.5 Защо тази теорема е "сърцето" на всичко

Теоремата казва нещо много мощно: **за всяко "стабилно" частично решение A и всеки срез, който не пресича A , най-лекото прекосяващо ребро е безопасен избор.** Локално оптималното решение (по този срез) е и глобално допустимо.

Това превръща задачата от "търси МПД глобално" в "избирай поредица локални леки ребра". Двата алгоритъма по-долу се различават **само в това как избират среза**.

3. Алгоритъм на Prim

3.1 Интуиция

Алгоритъмът на Prim прилича на **обхождане в ширина (BFS)**, но вместо да избира съсед по реда на пристигане, избира **най-лекия** наличен.

Започваме от един стартов връх s . Имаме "дърво в процес на изграждане" T . На всяка стъпка имаме срез между **вече включените** върхове $V(T)$ и **още невключените** $V \setminus V(T)$. Намираме най-леко ребро, прекосяващо този срез, и присъединяваме съответния връх към дървото.

*Във всеки момент A образува **едно единствено дърво** — точно това е разликата с *Kruskal*, където A може да е гора от много дървета.*

3.2 Общ псевдокод (Prim generic)

```
Prim_generic(G = (V, E), w, s):
1  V(T) ← {s}
2  A ← ∅
3  while V \ V(T) ≠ ∅ do
4      намери леко ребро e = (x, y), прекосяващо среза {V(T), V \ V(T)}
5      нека x ∈ V(T)          // y ∉ V(T)
6      V(T) ← V(T) ∪ {y}
7      A ← A ∪ {e}
8  return A
```

Коректност (накратко): Във всеки момент A е подмножество на ребрата на някое МПД (инвариант). Срезът $\{V(T), V \setminus V(T)\}$ винаги е съобразен с A , защото и двата края на всяко ребро от A са в $V(T)$. По МПД-теоремата, леко прекосяващо ребро е сигурно за

A . Следователно $A \cup \{e\}$ също е подмножество на ребрата на някое МПД. Алгоритъмът свършва, когато $V(T) = V$, тоест когато A е ПД — а понеже е винаги подмножество на МПД, A е самото МПД.

3.3 Защо наивната реализация е бавна?

Цикълът се изпълнява $n - 1$ пъти. Ако на всяка итерация прегледаме всички m ребра, за да намерим леко прекосяващо ребро, получаваме $\Theta(nm)$ — кубично в най-лошия случай. Дори ако гледаме само ребрата, прекосяващи среза, в най-лошия случай (пълнен граф) сумата $\sum_{k=1}^{n-1} k(n - k) = \Theta(n^3)$. Това е неприемливо.

Идеята за подобрене: на всяка итерация *не търсим леко ребро от нула*, а **поддържаме информация**, която позволява бърз избор.

3.4 Базов вариант на Prim — $\Theta(n^2)$

Класифицираме върховете на три вида:

- **Дървесни** ($V(T)$): вече включени в дървото. Не правим нищо с тях.
- **Гранични** (fringe): не са в дървото, но са съседни на поне един дървесен връх. За всеки такъв връх u поддържаме **кандидат-ребро** — най-лекото ребро между u и $V(T)$, заедно с теглото му (**гранично тегло**).
- **Неизвестни** (unseen): не са в дървото и нямат съсед в дървото.

Идея: наместо да търсим леко прекосяващо ребро (което би изисквало да обикаляме много ребра), търсим **граничен връх с минимално гранично тегло** — а реброто му е готово в $\Theta(1)$.

```

Prim1(G = (V, E), w, s):
1  V(T) ← {s}
2  A ← ∅
3  foreach u ∈ V
4      status[u] ← неизвестен
5  status[s] ← дървесен
6  x ← s
7  while V \ V(T) ≠ ∅ do
8      // x е последно прибавеният връх — обновяваме съседите му
9      foreach y ∈ adj[x]
10         if status[y] = граничен
11             if w((x, y)) < гранично_тегло[y]
12                 кандидат[y] ← (x, y)
13                 гранично_тегло[y] ← w((x, y))
14         if status[y] = неизвестен
15             status[y] ← граничен
16             кандидат[y] ← (x, y)
17             гранично_тегло[y] ← w((x, y))
18     намери граничен връх y* с минимално гранично_тегло
19     e ← кандидат[y*]
20     x ← y*
21     status[x] ← дървесен
22     V(T) ← V(T) ∪ {x}
23     A ← A ∪ {e}
24 return A

```

Сложност ($\Theta(n^2)$):

- **Намиране на минимален граничен връх (ред 18):** на всяка итерация преглеждаме всички гранични върхове — в най-лошия случай $\Theta(n)$. Общо: $\Theta(n - 1) + \Theta(n - 2) + \dots + \Theta(1) = \Theta(n^2)$.
- **Обхождане на списъците на съседство (ред 9):** общата сума за цялото изпълнение е $\Theta(n + m)$, защото всеки списък се обхожда веднъж — когато върхът стане дървесен.

Общо: $\Theta(n^2 + n + m) = \Theta(n^2)$, защото $m \leq \binom{n}{2} = O(n^2)$.

***Любопитно:** За **гъсти графи** ($m \sim n^2$) тази версия е оптимална — самите списъци на съседство имат размер $\Theta(n^2)$, така че нищо не може да е по-добро.*

3.5 Изтънчен вариант на Prim с приоритетна опашка — $\Theta(m \log n)$

За **разредени графи** (m е близо до n) можем да направим по-добре, ако използваме **приоритетна опашка** (min-heap).

Идея: държим всички върхове извън дървото в приоритетна опашка Q , като ключът на връх u е минималното тегло на ребро между u и текущото дърво (или ∞ , ако такава няма). Тогава "извади минимален граничен връх" става **Extract-Min**, което при двоична пирамида е $\Theta(\log n)$.

Прости ли се това с по-удобна нотация: $u.\text{key}$ = текущ ключ на u , $\pi[u]$ = родителят на u в строящото се дърво.

```
Prim2(G = (V, E), w, s):
1  foreach u ∈ V
2      u.key ← ∞
3      π[u] ← Nil
4  s.key ← 0
5  създай празна min приоритетна опашка Q
6  Q ← V // Build-Heap, Θ(n)
7  while not isempty(Q) do
8      x ← Extract-Min(Q)
9      foreach y ∈ adj[x]
10         if y ∈ Q and w((x, y)) < y.key
11             y.key ← w((x, y)) // Decrease-Key
12             π[y] ← x
13  return π[1..n]
```

Три важни имплементационни детайла:

1. **Build-Heap (ред 6)** работи в $\Theta(n)$.
2. **Проверката $y \in Q$ (ред 10)** се прави в $\Theta(1)$ с булев флаг за всеки връх — `True` когато е в опашката, `False` когато е извадена.
3. **Намаляването на ключ (ред 11) не е** обикновено присвояване — то е **Decrease-Key** в пирамидата, което за двоична пирамида работи в $\Theta(\log n)$.

Сложност (двоична пирамида):

Операция	Брой	Сложност за всяка	Обща
Build-Heap	1	$\Theta(n)$	$\Theta(n)$
Extract-Min	n	$\Theta(\log n)$	$\Theta(n \log n)$
Decrease-Key	до m (по веднъж за всяко ребро)	$\Theta(\log n)$	$\Theta(m \log n)$

Обща сложност: $\Theta(n + n \log n + m \log n) = \Theta(m \log n)$ (за свързан граф $m \geq n - 1$).

Внимание: За **гъсти** графи ($m \sim n^2$) това е $\Theta(n^2 \log n)$ — **по-зле** от базовия вариант ($\Theta(n^2)$)! Изтънченият вариант е по-добър само за **разредени** графи.

3.6 Със пирамида на Fibonacci — $\Theta(m + n \log n)$

При пирамидите на Fibonacci:

- **Decrease-Key** работи в **амортизирано** $\Theta(1)$.
- **Extract-Min** остава $\Theta(\log n)$.

Тогава:

- Всички Decrease-Key операции: $\Theta(m)$ (амортизирано).
- Всички Extract-Min операции: $\Theta(n \log n)$.

Обща сложност: $\Theta(m + n \log n)$.

Кога си струва? Само за **много големи разредени графи**. Скрытите константи в пирамидите на Fibonacci са големи, и в практиката за повечето размери двоичните пирамиди печелят.

3.7 Обобщена таблица за сложността на Prim

Структура за приоритетите	Сложност
Масив (наивно търсене на мин)	$\Theta(n^2)$ — добре за гъсти графи
Двоична пирамида	$\Theta(m \log n)$ — добре за разредени графи
Пирамида на Fibonacci	$\Theta(m + n \log n)$ — асимптотически най-добре

4. Алгоритъм на Kruskal

4.1 Интуиция

Идеята е съвсем различна от Prim:

1. **Сортираме ребрата по тегло** в намаляващ ред.
2. Започваме с **гора** от n тривиални дървета — по едно за всеки връх.
3. Минаваме през сортираните ребра в реда им. За всяко ребро:
 - Ако двата му края са в **различни дървета** на гората → добавяме реброто; то слива двете дървета.
 - Ако са в **едно и също дърво** → пропускаме реброто (би образувало цикъл).
4. Спираме, когато гората има точно едно дърво — то е МПД.

***Срезът тук е "имплицитен":** когато гледаме поредното ребро $e = (u, v)$ и краищата му са в различни дървета T_1 и T_2 , срезът $\{V(T_1), V \setminus V(T_1)\}$ е съобразен с текущата гора, и e е леко за този срез (защото всички ребра преди него вече са разгледани и или са в гората, или биха създали цикъл).*

4.2 Общ псевдокод (Kruskal generic)

```
Kruskal_generic( $G = (V, \{e_1, \dots, e_m\})$ ,  $w$ ):  
1  сортирай ребрата по тегла в редица  $\langle e_1, e_2, \dots, e_m \rangle$   
2   $F \leftarrow \{(\{v\}, \emptyset) \mid v \in V\}$            // гора от тривиални дървета  
3   $i \leftarrow 1$   
4  while  $F$  съдържа повече от едно дърво do  
5      нека  $e_i$  има краища  $u$  и  $v$   
6      if  $u$  и  $v$  са в различни дървета  $T'$  и  $T''$  на  $F$   
7          слей  $T'$  и  $T''$  чрез  $e_i$  в едно дърво в  $F$   
8       $i++$   
9  return единственото дърво в  $F$ 
```

4.3 Коректност (трите доказателства)

Коректността на алчен алгоритъм се състои от **три тези**:

1. Алгоритъмът **терминира**.

2. Изходът е **ПД** (допустимо решение).
3. Изходът е **МПД** (оптимално решение).

4.3.1 Лема 43: F е покриваща гора през цялото време

При всяко достигане на ред 4, F е покриваща гора на G .

Доказателство: По индукция. В началото F се състои от тривиални дървета — гора. На всяка итерация или нищо не променяме (ред 6 е лъжа), или сливаме две дървета (ред 7), което пак е гора. ■

4.3.2 Теорема 96: Kruskal_generic терминира

Идея: Трябва да докажем, че рано или късно F ще има само едно дърво.

Инвариант 29: При всяко достигане на ред 4, за всяко ПД T , такова че $E(F) \subseteq E(T)$, е вярно, че: $E(T) \setminus E(F) \subseteq \{e_i, e_{i+1}, \dots, e_m\}$

С други думи: "още не съм видял" ребрата, които трябва да добавя.

Защо това гарантира терминация? Ако F има $k \geq 2$ дървета, то $|E(F)| = n - k$, а за всяко ПД T : $|E(T)| = n - 1$, тоест $|E(T) \setminus E(F)| = k - 1$ ребра, всички в "оставащата" редица. Алгоритъмът ще стигне до тях и ще ги използва за сливания. ■

4.3.3 Теорема 97: Kruskal_generic връща ПД

Следва веднага от Теорема 96: при последното достигане на ред 4, F има само едно дърво, а от Лема 43 знаем, че F е покриваща гора. Покриваща гора с едно дърво е покриващо дърво. ■

4.3.4 Теорема 98: Kruskal_generic връща МПД

Тук влиза в действие сортирането!

Нека $\mathcal{M}$ е множеството на всички МПД на G , и $\mathcal{M}_F = \{T \in \mathcal{M} : E(F) \subseteq E(T)\}$.

Инвариант 30: При всяко достигане на ред 4, $\mathcal{M}_F \neq \emptyset$ — съществува МПД, съдържащо ребрата на текущата гора.

База. В началото $E(F) = \emptyset$, и поне едно МПД съществува (графът е свързан). ✓

Поддръжка. Допускаме, че $E(F) \subseteq E(T)$ за някое МПД T . Разглеждаме итерация в момент i :

- Ако условието на ред 6 е лъжа, F не се променя. Инвариантът остава. ✓

- Ако е истина, разглеждаме среза $\{V(T'), V \setminus V(T')\}$, където T' е дървото в F , съдържащо единия край на e_i . Този срез е **съобразен с $E(F)$** (защото всички ребра на F са вътре в дървета). Освен това e_i е **леко прекосяващо ребро** — защото всички по-леки ребра $e_1, \dots, e_{i-1}$ или вече са в F , или са били пропуснати (защото и двата им края са били в едно дърво на F). По Теорема 94, e_i е сигурно за $E(F)$. Тогава $E(F) \cup \{e_i\}$ е подмножество на ребрата на някое МПД. ✓

Терминация. При последното достигане на ред 4, F има едно дърво, чиито ребра са подмножество на ребрата на някое МПД. Но самото дърво е ПД — с $n - 1$ ребра. Едно МПД също има $n - 1$ ребра. Следователно F е МПД. ■

4.4 Сложност на Kruskal — защо наивно е бавно

Наивна реализация:

- **Сортиране:** $\Theta(m \log m) = \Theta(m \log n)$ (защото $m \leq n^2 \implies \log m \leq 2 \log n$).
- **Проверка дали добавянето на ребро образува цикъл:** ако правим с BFS/DFS върху текущата гора, всяка проверка отнема $\Theta(n)$ в най-лошия случай.
- В най-лошия случай преглеждаме всички m ребра (например когато най-тежкото ребро е мост — задължително участва в МПД).

Общо: $\Theta(m \log n) + \Theta(nm) = \Theta(nm)$. За гъсти графи това е $\Theta(n^3)$ — **кубично**.

Решението: ефикасна **Union-Find** структура от данни, която поддържа разбиване на върховете на компоненти и позволява бързо да:

- проверим дали два върха са в една компонента;
- сливаме две компоненти.

5. Union-Find структури от данни

5.1 Задачата

Имаме опорно множество $V = \{1, 2, \dots, n\}$. Искаме да поддържаме **разбиване** на V , поддържащо две операции:

- **Find(u):** връща идентификатор на дяла, в който се намира u .
- **Union(i, j):** слива дяловете с идентификатори i и j .

За да е ефикасен Kruskal, искаме **двете да работят в (амортизирано) $O(\log n)$ или по-бързо.**

5.2 Два крайни (наивни) подхода, които не работят

Подход А: всеки елемент носи името на дяла си.

Всеки връх е запис с поле "име на дяла". Тогава `Find` е $\Theta(1)$. Но `Union(ДЪБ, БУК)` иска да обновим името на всеки елемент в единия дял — $\Theta(n)$ в най-лошия случай.

Подход В: всеки дял е свързан списък.

`Find` би искало да намерим върха в кой списък е — $\Theta(n)$. Лошо за `Find`.

И двата подхода имат поне една бавна операция. **Балансираното решение е чрез дървета.**

5.3 Имплементация чрез ориентирана гора

Реализираме разбиването с **гора от ориентирани дървета**, в които:

- **Върховете** са елементите на V .
- Всеки връх u има **указател към родителя си** $\pi[u]$.
- Указателят на корена сочи към самия него: $\pi[\text{корен}] = \text{корен}$.
- **Лидерът** на дял е коренът на съответното дърво — той служи като идентификатор на дяла.

Псевдокод на Find:

```
Find(u):  
1  if u ≠ π[u]  
2      return Find(π[u])  
3  else  
4      return u
```

Просто "катерим се" нагоре по родителските указатели до корена.

Псевдокод на Union (наивен):

```
Union_generic(u, v):    // u и v са лидери на различни дялове
1  Нека x е единият от u и v (произволно)
2  Нека y е другият
3   $\pi[y] \leftarrow x$ 
```

Това е $\Theta(1)$ — само променяме един указател.

Проблем: ако правим Union-и произволно, височината на дървото може да стане **линейна** $\Theta(n)$, и тогава Find ще е линеен. Това е катастрофа.

***Пример за лошо Union:** ако винаги "наказваме" по-голямото дърво (правим корена му дете на корена на по-малкото), дървото бързо ще се източи в път.*

5.4 Евристика 1: Union by Rank

Идеята: "наказваме" по-малкото дърво — правим корена му дете на корена на по-голямото. Така минимизираме увеличаването на височината.

Вариант 1: рангът е броят на върховете.

```
Union_by_rank_v1(u, v):
1  if rank(u)  $\geq$  rank(v)
2       $\pi[v] \leftarrow u$ 
3      rank(u)  $\leftarrow$  rank(u) + rank(v)
4  else
5       $\pi[u] \leftarrow v$ 
6      rank(v)  $\leftarrow$  rank(v) + rank(u)
```

Начален ранг на всеки връх е 1.

Вариант 2: рангът е височината.

```

Union_by_rank_v2(u, v):
1  hu ← rank(u); hv ← rank(v)
2  if hu > hv
3      π[v] ← u
4  else if hu < hv
5      π[u] ← v
6  else
7      // равни височини — височината расте с 1
8      π[v] ← u
9      rank(u) ← hu + 1

```

Начален ранг е 0 (тривиалните дървета имат височина 0).

Теорема 99: Височина $O(\log n)$ при Union by Rank

Твърдение: След произволна серия от Union_by_rank операции, всяко дърво има височина $h = O(\log n)$, където n е броят на върховете в това дърво.

Доказателство (по индукция по h):

Твърдение: $\exists c > 0 : h \leq c \log n$.

База ($h = 0$): $n = 1$ (единичен връх). За всяка $c > 0$, $0 \leq c \log 1 = 0$. ✓

Стъпка: Допускаме, че за дървета с височина h е вярно $h \leq c \log n$. Разглеждаме дърво с височина $h + 1$, получено от сливането на две дървета. **Това може да стане само ако и двете изходни дървета са с височина точно h** — иначе при Union by rank височината не би се увеличила.

Нека двете дървета имат n_1 и n_2 върха, и БОО $n_1 \geq n_2$. Тогава $n_1 + n_2 \geq 2n_2$. По индуктивно предположение: $c \log(n_1 + n_2) \geq c \log(2n_2) = c \log n_2 + c \geq h + c \geq h + 1$ за $c = 1$. ■

Следствие: Find работи в $O(\log n)$, и Kruskal с Union by Rank има сложност $\Theta(m \log n)$ — оптимална, защото само сортирането вече иска $\Omega(m \log n)$.

5.5 Евристика 2: Path Compression

Идеята: при всяко Find, докато "катерим" нагоре до корена, **правим всеки минат връх дете директно на корена**. Това "сплеска" дървото и прави бъдещите Find много по-бързи.

```
Find_with_path_compression(u):
1  if u ≠ π[u]
2      π[u] ← Find_with_path_compression(π[u])
3  return π[u]
```

Пример:

Преди Find(17):	След Find(17):
1	1
	/ / / / \
2	17 13 6 2 ...
6	18 14 7 ...
	15
13	16
17	(всички предци на 17
	стават деца на 1)
18	

5.6 Сложност с двете евристики едновременно

Теорема: При m операции Union или Find върху n -елементно опорно множество, общата сложност е: $O(m \cdot \alpha(n))$ където $\alpha(n)$ е **обратната функция на Акерман** — изключително бавно растяща функция.

За практически цели: $\alpha(n) \leq 4$ за всяко n , което може да възникне в реалния свят (буквално за n по-малко от броя атоми във Вселената). Следователно операциите са **практически $O(1)$ амортизирано**.

5.7 Прилагане към Kruskal

Финалният ефикасен псевдокод на Kruskal с Union-Find:

```

Kruskal(G = ({1, ..., n}, E), w):
1  A ← ∅
2  направи разбиването { {1}, {2}, ..., {n} } // тривиална гора
3  сортирай E по тегла
4  count ← 0
5  while count < n - 1 do
6      нека e = (u, v) е следващото ребро в сортираната редица
7      if Find(u) ≠ Find(v)
8          A ← A ∪ {e}
9          Union(Find(u), Find(v))
10         count++
11  return A

```

Сложност:

- **Сортиране (ред 3):** $\Theta(m \log m) = \Theta(m \log n)$.
- **while-цикъл:** до m итерации, всяка от които прави $O(1)$ Find и Union операции.
Обща сложност на всички Union/Find: $O(m \cdot \alpha(n))$ — практически $O(m)$.

Обща сложност: $\Theta(m \log n)$.

Заклучение: *Kruskal* е асимптотически точно толкова бърз, колкото и *Prim* с двоична пирамида — и двата $\Theta(m \log n)$. На практика **Kruskal често е по-бърз на разредени графи** (защото сложността му е "доминирана" от сортирането, а сортирането в практиката е извънредно бързо), а **Prim е по-добър на гъсти графи** (особено базовата версия с $\Theta(n^2)$).

6. Сравнителна таблица

Аспект	Prim (масив)	Prim (двоична пирамида)	Prim (Fibonacci)	Kruskal (Union-Find)
Сложност	$\Theta(n^2)$	$\Theta(m \log n)$	$\Theta(m + n \log n)$	$\Theta(m \log n)$
Подходящ за	гъсти графи	разредени	много големи разредени	разредени
Идея	расте един срез от s	приоритетна опашка	приоритетна опашка	сортирани ребра + DSU
Сложност в практиката	$O(1)$ константи	средно бърз	големи константи	константи в сортирането

7. Интуитивно обобщение

Какво е общото между Prim и Kruskal? И двата:

- Изграждат МПД итеративно, добавяйки по едно ребро.
- Винаги добавят **леко прекосяващо ребро на някакъв срез** — затова и двата са коректни по МПД-теоремата.
- Във всеки момент частичното решение е подмножество на ребрата на някое МПД.

В какво се различават?

- Prim** поддържа **едно дърво**, растящо от стартов връх. Срезът е винаги {дървесни, останалите}.
- Kruskal** поддържа **гора от много дървета**. Срезът е имплицитен — определя се от компонентите.

Защо ни е МПД-теоремата? Тя превръща глобална оптимизация в поредица локални избори. Това е същината на алчните алгоритми за МПД.

Защо ни е Union-Find? Защото без нея проверката "от едно дърво ли са?" в Kruskal е линейна, а с нея — практически константна.

Приложение: Бърза справка с теореми и инварианти

Обект	Какво казва
Опр. 91 (сигурно ребро)	e е сигурно за A , ако $A \cup \{e\}$ е подмножество на МПД
Опр. 92 (срез, леко ребро)	срез = разбиване на V на 2 дяла; леко ребро = мин. тегло сред прекосяващите
Т. 94 (МПД-теорема)	леко ребро, прекосяващо срез, съобразен с $A \rightarrow$ сигурно за A
Инв. на Prim	при всяко достигане на цикъла, A е МПД на подграфа, индуциран от $V(T)$
Инв. 29 (Kruskal)	ребрата на всяко МПД, неprisъстващи в F , са измежду още непрегледаните
Инв. 30 (Kruskal)	съществува МПД, съдържащо текущите ребра на F
Т. 99 (Union by Rank)	височината на дървото в DSU е $O(\log n)$
$\alpha(n)$ (Акерман)	при двете евристики амортизираната сложност е $O(\alpha(n)) \approx O(1)$